

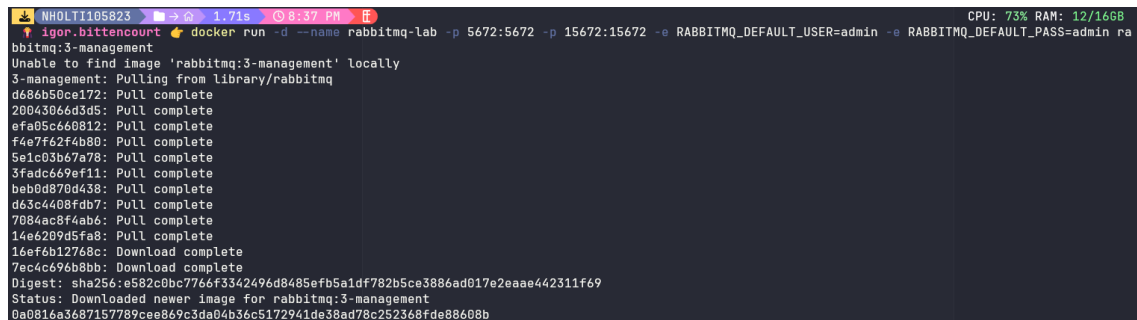
Atividade 04 - Laboratório sobre RabbitMQ

Igor de Oliveira Bittencourt Moreira

20231012000959

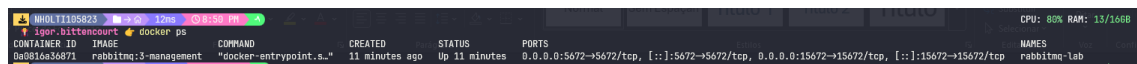
1. Subir RabbitMQ com Docker

a. Subir o docker `docker run -d --name rabbitmq-lab -p 5672:5672 -p 15672:15672 -e RABBITMQ_DEFAULT_USER=admin -e RABBITMQ_DEFAULT_PASS=admin rabbitmq:3-management`



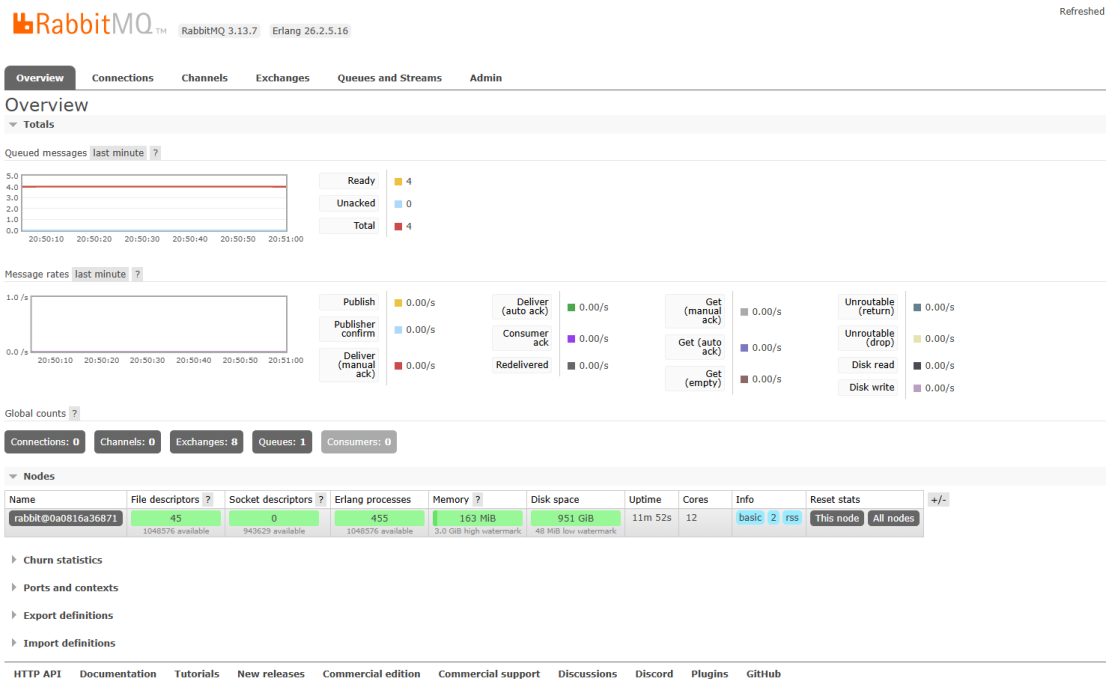
```
igor.bittencourt@igor: ~$ docker run -d --name rabbitmq-lab -p 5672:5672 -p 15672:15672 -e RABBITMQ_DEFAULT_USER=admin -e RABBITMQ_DEFAULT_PASS=admin rabbitmq:3-management
Unable to find image 'rabbitmq:3-management' locally
3-management: Pulling from library/rabbitmq
d686b50ce172: Pull complete
20043066d3d5: Pull complete
efa05c660812: Pull complete
f4e7f62f4b80: Pull complete
5e1c03b67a78: Pull complete
3fad669ef11: Pull complete
beb0d870d438: Pull complete
d63c4408fdb7: Pull complete
7084ac8f4ab6: Pull complete
14e6209d5fab: Pull complete
16ef6b12768c: Download complete
7ec4c696b8bb: Download complete
Digest: sha256:e582c0bc7766f3342496d8485efb5a1df782b5ce3886ad017e2eaae442311f69
Status: Downloaded newer image for rabbitmq:3-management
0a0816a3687157789cee869c3da04b36c5172941de38ad78c252368fde88608b
```

b. Verificar docker os



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0a0816a36871	rabbitmq:3-management	"docker-entrypoint.s..."	11 minutes ago	Up 11 minutes	0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp	rabbitmq-lab

2. Acessar interface web <http://localhost:15672> Login: usuário: admin senha: admin a. Explore/visualize as abas na tela inicial Overview Connections Channels Exchanges Queues



3. Criar uma Queue a. Vá em: Queues, Add a new queue b. Preencha: Name: fila.lab Durable: (marcado) c. Clique em Add queue d. Você criou uma fila persistente (Durable). Pesquise o que é uma fila persistente e coloque sua resposta aqui.

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Queues

▼ All queues (1)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Overview						Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack		
/	fila.lab	classic	D	running	4	0	4	0.00/s	0.00/s	0.00/s		

4. Criar um Exchange

a. Pesquise sobre o que é uma Exchange no RabbitMQ. Cite suas vantagens.

Uma Exchange no RabbitMQ é o componente responsável por receber as mensagens enviadas pelos produtores e decidir para qual fila elas devem ser encaminhadas. Em vez de enviar diretamente para uma fila, o produtor envia a mensagem para uma Exchange, que aplica regras de roteamento para distribuir essa mensagem corretamente.

Esse funcionamento traz algumas vantagens importantes. A principal é o desacoplamento, pois o produtor não precisa conhecer as filas ou os consumidores. Também oferece mais flexibilidade, já que é possível alterar as regras de roteamento sem mudar o código de quem envia as mensagens. Além disso, permite maior escalabilidade, pois uma mesma mensagem pode ser distribuída para várias filas e consumida por diferentes processos. Por fim, melhora a organização do sistema, separando claramente a responsabilidade de envio da lógica de distribuição das mensagens.

- b. Crie uma exchange: Exchanges, Add a new exchange c. Preencha: Name: exchange.lab Type: direct Durable: (marcado) d. Clique em Add exchange

Overview Connections Channels **Exchanges** Queues and Streams Admin

Exchanges

▼ All exchanges (8)

Pagination

Page of 1 - Filter: ☐ Regex ?

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.trace	topic	D I			
/	amq.topic	topic	D			
/	 exchange.lab	direct	D	0.00/s	0.00/s	

▼ Add a new exchange

- e. Quais são os types disponíveis para uma exchange? Explique cada um deles.

1. Direct

Envia a mensagem para a fila cuja *routing key* é exatamente igual à definida no binding. É um roteamento direto e preciso, usado quando você sabe exatamente para onde a mensagem deve ir.

2. Fanout

Ignora a *routing key* e envia a mensagem para todas as filas ligadas à Exchange. É útil quando a mesma mensagem precisa ser entregue para vários consumidores ao mesmo tempo.

3. Topic

Usa padrões na *routing key*, permitindo roteamento mais flexível. É possível usar curingas como * (uma palavra) e # (várias palavras). Ideal para cenários onde as mensagens são categorizadas, como logs ou eventos.

4. Headers

Não usa *routing key*. O roteamento é feito com base em atributos (headers) da mensagem. É mais avançado e permite regras mais específicas, mas costuma ser menos utilizado por ser mais complexo.

5. Criar Binding Conecte o exchange à uma fila a. Clique em exchange.lab b. Vá até Bindings c. Em "Add binding" e preencha: To queue: fila.lab Routing key: teste d. Clique em Bind

Overview Connections Channels **Exchanges** Queues and Streams Admin

Exchange: **exchange.lab**

▼ Overview

Message rates last minute ?

1.0 /s

0.0 /s

20:56:20 20:56:30 20:56:40 20:56:50 20:57:00 20:57:10

Publish (In) 0.00/s

Publish (Out) 0.00/s

Details

Type direct

Features durable: true

Policy

▼ Bindings

This exchange

⇕

To	Routing key	Arguments	
fila.lab	teste		Unbind

e. Para que é utilizado o Routing Key? Pesquise e descreva o que entendeu.

1. A **Routing Key** no RabbitMQ é uma informação (uma “chave”) enviada junto com a mensagem e usada pela Exchange para decidir para qual fila ela deve ser encaminhada. Ela funciona como um critério de roteamento. Quando você cria um binding entre uma Exchange e uma fila, você define uma Routing Key. Depois, quando uma mensagem chega na Exchange com essa mesma chave (ou um padrão compatível, dependendo do tipo de exchange), ela é direcionada para a fila correspondente. Em resumo, a Routing Key serve para controlar e filtrar o destino das mensagens, garantindo que cada uma vá apenas para as filas corretas conforme as regras definidas.

6. Publicar mensagem a. Ainda dentro do exchange.lab, vá em Publish message b. Preencha: Routing key: teste Payload: { "remetente": "eu@ferreira.br", "mensagem": "Olá RabbitMQ, aqui sou eu!!!" } c. Clique em Publish message d. Repita esses passos e publique mais 5 mensagens. Todas com Routing key igual a teste.

Overview Connections Channels **Exchanges** Queues and Streams Admin

Policy

▼ Bindings

This exchange

↓

To	Routing key	Arguments
file.lab	teste	Unbind

Message published.

Close

Add binding from this exchange

To queue: *

Routing key:

Arguments: = String ▼

Bind

▼ Publish message

Routing key: teste

Headers: ? = String ▼

Properties: ? =

Payload: { "remetente": "eu@igor.br", "mensagem": "Olá RabbitMQ, aqui é o Igor lindo!!!" }

Payload encoding: String (default) ▼

Publish message

► Delete this exchange

7. Consumir mensagem a. Vá em Queues b. Clique em fila.lab c. Role até Get messages d. Configure: Ack mode: auto ack Messages: 1 e. Clique em Get Message(s) f. Repita os dois últimos passos, agora com Get Messages(s) igual a 3


RabbitMQ 3.13.7 Erlang 26.2.5.16

Refreshed 2026-03-18 21:02:03
Refresh every 5 seconds

Virtual host /
Cluster rabbit@0a0816a36871
User admin
Log out

Overview
Connections
Channels
Exchanges
Queues and Streams
Admin

Queue fila.lab

Overview

Queued messages last minute



Ready 5
Unacked 0
Total 5

Message rates last minute



Publish 0.00/s
Consumer ack 0.00/s
Get (auto ack) 0.00/s
Deliver (manual ack) 0.00/s
Redelivered 0.00/s
Get (empty) 0.00/s
Deliver (auto ack) 0.00/s
Get (manual ack) 0.00/s

Details

Features	queue storage version: 1	State idle	Total 5	Ready 5	Unacked 0	In memory 4	Persistent 5	Transient, Paged Out 0
Policy		Consumers 0	Messages 7	428 B	428 B	0 B	345 B	428 B
Operator policy		Consumer capacity 0%	Message body bytes 7					0 B
Effective policy definition			Process memory 7	14 KiB				

Consumers (0)

Bindings (2)

From

Routing key / Arguments

(Default exchange binding)

exchange.lab

teste

Unbind

This queue

Add binding to this queue

From exchange:

Routing key:

Arguments:

=

String

Bind

Publish message

Get messages

Warning: getting messages from a queue is a destructive action.

Ack Mode: Nack message request true

Encoding: Auto string / base64

Messages: 3

Get Message(s)

Message 1

The server reported 4 messages remaining.

Exchange

exchange.lab

Routing Key

teste

Redelivered

0

Properties

delivery_mode: 2

Payload

60 bytes

Encoding: string

```
{
  "resetente": "eu@ferreira.br",
  "message": "Olá RabbitMQ, aqui sou eu 31111"
}
```

Message 2

The server reported 3 messages remaining.

Exchange

exchange.lab

Routing Key

teste

Redelivered

0

Properties

delivery_mode: 2

Payload

60 bytes

Encoding: string

```
{
  "resetente": "eu@ferreira.br",
  "message": "Olá RabbitMQ, aqui sou eu 31111"
}
```

Message 3

The server reported 2 messages remaining.

Exchange

exchange.lab

Routing Key

teste

Redelivered

0

Properties

delivery_mode: 2

Payload

60 bytes

Encoding: string

```
{
  "resetente": "eu@ferreira.br",
  "message": "Olá RabbitMQ, aqui sou eu 41111"
}
```

Move messages

Delete

Purge

Runtime Metrics (Advanced)

[HTTP API](#)
[Documentation](#)
[Tutorials](#)
[New releases](#)
[Commercial edition](#)
[Commercial support](#)
[Discussions](#)
[Discord](#)
[Plugins](#)
[GitHub](#)

8. Experimento com roteamento incorreto Use routing key inválida: a. Dentro do exchange.lab, vá em Publish message e preencha: Routing key: errada Payload: {

"remetente": "novoeu@ferreira.br", "mensagem": "Hey, ainda sou o RabbitMQ!" } b. Clique em Publish message c. Repita esses passos e publique mais 2 mensagens. Todas com Routing key igual a errada.

The screenshot shows the RabbitMQ web interface. At the top, under 'Bindings', there is a table with one binding:

To	Routing key	Arguments
fila.lab	teste	

Below the table is a 'Bind' button. To the right, a yellow message box says 'Message published, but not routed.' with a 'Close' button. Below this, the 'Add binding from this exchange' section has fields for 'To queue', 'Routing key', and 'Arguments'. The 'Publish message' section is active, showing 'Routing key: errado', 'Headers: ?', 'Properties: ?', and a 'Payload' field containing a JSON object:

```
{  "remetente": "novoeu@ferreira.br",  "mensagem": "Hey, ainda sou o RabbitMQ!"}
```

 The 'Payload encoding' is set to 'String (default)'. At the bottom, there is a 'Publish message' button.

d. A mensagem chegou à fila? Por que não?

Não chegou. Isso acontece porque a *routing key* usada ("errada") não corresponde à chave definida no binding ("teste"). Como a Exchange não encontrou nenhuma regra de roteamento compatível, ela não soube para qual fila enviar a mensagem e simplesmente a descartou (ou não roteou para nenhuma fila).

e. Posso criar outros bindings e associá-los a outras filas? Onde isso é útil?

Sim, é possível criar vários bindings ligando a mesma Exchange a diferentes filas, cada um com uma *routing key* diferente. Isso é útil em cenários onde você precisa distribuir mensagens para destinos distintos com base em regras, como separar tipos de eventos (ex: pedidos, pagamentos, logs), direcionar mensagens para diferentes sistemas ou permitir que múltiplos consumidores recebam apenas os dados relevantes para eles.